PAYMENT SYSTEM

A Project Report Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By

J. MAMATHA	**22221A0535**
K.N.S. NARASIMHA KUMAR	**22221A0557**
K. SAILAJA	**22221A0543**
A. SANDHYA	**22221A0506**
Y. DURGAVENKAT	**22221A05C6**

Under the Esteemed Guidance of

Mr. T. V. L. SRINIVAS

M. Tech

Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAM VENKATA CHALAMAYYA ENGINEERING COLLEGE

(AUTONOMOUS)

(Approved by A.I.C.T.E, New Delhi & Permanently Affiliated to J. N.T.U.K, Kakinada)

(Accredited by N.B.A & NAAC with 'A' Grade)

ODALAREVU – 533210

2022-2026

BONAM VENKATA CHALAMAYYA ENGINEERING COLLEGE

(AUTONOMOUS)

(Approved by A.I.C.T.E, New Delhi & Permanently Affiliated to J. N.T.U.K, Kakinada)

(Accredited by N.B.A & NAAC with 'A' Grade)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project work entitled **"PAYMENT SYSTEM USING SPRING BOOT, JSP AND MYSQL"** is being submitted for the partial fulfilment of the requirements for the award Of the degree of **Bachelor of Technology in Computer Science and Engineering, at BVC Engineering College, Odalarevu**, is a bonafide work done by **J. MAMATHA (22221A0535), S. N. S. N. KUMAR (22221A0557), K. SAILAJA (22221A0543), A.SANDHYA(22-506), Y. DURGAVENKAT (22221A05C6)** under my guidance during the academic year 2025 - 2026 it has been found suitable for acceptance according to the requirement of the University.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree.

Project Guide
Mr. T. V. L. SRINIVAS
M. Tech
Professor

Head of the Department
Dr. MEERASHARIF SHEIK
M. Tech,Ph. D
Professor&HOD

External Examiner

ACKNOWLEDGEMENT

We profoundly grateful to express our deep sense of gratitude and respect towards our Guide Mr. T. V. L . SRINIVAS, M. Tech, Professor of CSE Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his excellent guidance from the selection of the project and his valuable suggestions through out the project work. His constant encouragement and support has been the cause of our success, in completing these thesis.

We also grateful to the Project Co-Ordinator Mr.M.V.K.SUBASH,M.Tech.,(Ph.D), Assistant Professor of CSE Department,Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his excellent guidance right from selection of the project and his valuable suggestions throughout the project work.

We really thankful to Dr. MEERASHARIF SHEIK, M. Tech, Ph. D, Professor and Head of the Department of Computer Science and Engineering for providing the laboratory facilities to the fullest extent as and when required and also for giving us the opportunity to carry out the project work in the college.

We also thankful to Dr. MAHESWAR DUTTA, M.E, Ph. D, Principal, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, for his sup port during and till the completion of the project. And also thankful to our MANAGEMENT, Bonam Venkata Chalamayya Engineering College (A), Odalarevu, for their support during and till the completion of the project.

We are thankful to all the TEACHING AND NON-TEACHING STAFF of Computer Science and Engineering Department, Bonam Venkata Chalamayya Engineering College(A), Odalarevu, and FRIENDS for their direct and indirect help provided to us in completing the project.

Last but not least, we own all our success to our FAMILY MEMBERS, CLASSMATES AND TEACHERS from our child hood, whose vision, love and inspiration made us to reach out for these glories.

PROJECT ASSOCIATES

J. MAMATHA	**22221A0535**
K.N.S. NARASIMHA KUMAR	**22221A0557**
K. SAILAJA	**22221A0543**
A. SANDHYA	**22221A0506**
Y. DURGAVENKAT	**22221A05C6**

ABSTRACT

The Payment System is a Java Full Stack web application designed to provide a secure, efficient, and user-friendly platform for managing online financial transactions between service providers and customers. The system enables users to register, log in securely, view available services,make payments,and track transaction history in real time.Service provider can manage their services, monitor payments received, and maintain customer records. The application is developed using Spring Boot for backend business logic, JSP for server – side presentation, Bootstrap for responsive user interface design, and MySQL for reliable data storage.

The proposed system addresses the limitations of traditional manual or semi-automated payment processes reducing errors, improving transaction speed, and enhancing transparency. It supports structured data handling, automated calculations, and secure payment workflows.

Overall, the Java Full Stack Payment System offers a robust solution for modern digital payment management. It is suitable for organizations, institutions, or service-based platforms that require accurate payment processing,effective record management,secure user interactions.

The system improves operational efficiency, minimizes manual intervention, and lays a strong foundation for future enhancement such as paymentgatewayintegration,reportgeneration, and advanced analytics.

LIST OF CONTENTS

CONTENTS **PAGE NO**

CHAPTER - 1 INTRODUCTION.....1

CHAPTER - 2 LITERATURE SURVEY 2

CHAPTER - 3 SYSTEM ANALYSIS.....3

3.1 Existing System 3

3.1.1 Drawbacks.....4

3.2 Proposed System.....5

3.2.1 Advantages.....6

CHAPTER – 4 REQUIREMENT ANALYSIS.....7

4.1 Hardware Requirements.....7

4.2 Software Requirements.....7

CHAPTER - 5 SOFTWARE DESIGN.....8

5.1 System Architecture.....8

5.2 UML Diagrams 8

5.2.1 Class Diagram.....9

5.2.2 Use Case Diagram.....9

5.2.3 Sequence Diagram 10

CHAPTER - 6 SYSTEM ENVIRONMENT 11

6.1 What is java..... 11

6.2 Advantages of java over Other Languages.....11

6.3 Disadvantages of java.....12

6.4 History of java.....12

6.5 What is spring boot.....13

6.6 what is jsp.....15

6.7 what is bootstrap.....17

6.8 Applications of java.....19

6.9 Python Development Steps.....22

CHAPTER – 7 MODULES USED IN PROJECT.....23

7.1 Install Python Step-by-Step in Windows or MAC.....26

7.2 How to Install Python on Windows and Mac 27

CHAPTER – 8 TESTING 34

8.1 System Test 34

8.2 Types Of Tests.....34

8.2.1 Unit testing.....34

8.2.2 Integrated Testing.....34

8.2.3 Functional test.....35
8.2.4 White Box Testing.....35
8.2.5 Black Box Testing.....36
8.2.6 Unit Testing.....36
CHAPTER - 9 IMPLEMENTATION.....38
9.1 Methodology.....38
9.2 Source Code.....40
CHAPTER – 10RESULTS.....55
CHAPTER – 11 CONCLUSION.....63
CHAPTER – 12 REFERENCES.....64

LIST OF FIGURES

FIGURES **PAGE NO**

Fig 5.1 System Architecture.....8
Fig 5.2 Class Diagram.....9
Fig 5.3 Use Case Diagram 9
Fig 5.4 Sequence Diagram.....10
Fig 10.1 Home (Register and Login) …….....55
Fig 10.2 Register page…… 56
Fig 10.3 Login page…….....57
Fig 10.4 Customer Dashboard.....57
Fig 10.5 Customer profile…….....58
Fig 10.6 My orders…….....58
Fig 10.7 Order Receipt….....59
Fig 10.8 All billers…….....59
Fig 10.9 Registered billers…….....60
Fig 10.10 Biller dashboard…….....60
Fig 10.11 Add product……..... 61
Fig 10.12 View products…….....61
Fig 10.13 All orders……...62
Fig 10.14 Customer orders.....62

CHAPTER - 1
INTRODUCTION

A Payment System is a vital part of modern digital applications, enabling secure and efficient electronic transactions between customers and service providers. With the rapid growth of online services, the demand for automated payment solutions has increased significantly. Traditional manual payment methods are time - consuming, error-prone, and lack transparency. A computerized payment system overcomes these limitations by automating transactions, maintaining centralized records, and ensuring faster and more accurate payment processing.

The Java Full Stack Payment System provides an end-to-end solution by integrating frontend and backend technologies. The frontend offers a user - friendly interface for customers to view services, make payments,and track transaction history,while service providers can manage service and monitor payments. The backend,developed using Spring Boot,handles authentication,business logic,and session management, while MySQL ensures secure and structured data storage. This layered architecture improves scalability, reliability, and maintainability.

Security and transparency are key features of the Payment System. Mechanisms such as user authentication, session tracking, and controlled access protect sensitive data. The system reduces manual effort,improves operational efficiency,and supports future enhancements like payment gateway integration and detailed reporting, making it a reliable solution for digital payment management.

CHAPTER - 2
LITERATURE SURVEY

The evolution of payment systems has been widely studied due to the rapid growth of digital transactions and online services. Early payment systems were mainly manual or semi - automated, relying on physical cash, checks, and basic record-keeping methods, which often result in errors and delays. Researchers highlighted the need for computerized systems to improve the accuracy, speed, and the reliability in financial transactions. With the information technology, electronic payment systems emerged, enabling users to perform transactions through web-based and mobile platforms.

Several studies focus on web-based payment systems developed using client–server architecture, where the frontend handles user interaction and the backend manages business logic and data storage. Technologies such as Java, JSP, and Spring frame works have been widely recommended due to their platform independence, security features and scalability.

Database systems like MySQL and Oracle are commonly used for storing transaction data securely and maintaining data integrity. Literature also emphasizes the importance of layered architecture to improve system maintainability and reduce complexity.

Security has been a major concern in payment system research. Many authors discuss mechanisms such as authentication, authorization, session management, and encryption to protect sensitive financial information. Recent studies highlight the integration of secure payment gateways, audit logs, and real-time transaction monitoring to prevent fraud.

CHAPTER - 3
SYSTEM ANALYSIS

System analysis is a crucial phase in software development that involves studying the existing system, identifying its limitations, and defining the requirements of the proposed system. This chapter provides a detailed analysis of the current payment process and highlights the need for a computerized payment system.

It also explains how the proposed Java Full Stack Payment System overcomes the drawbacks of the existing system and provides a secure, efficient, and scalable solution.

3.1 Existing System

The existing payment system mainly depends on manual or partially automated methods to manage transactions between customers and service providers. In many organizations and service-based platforms, payments are collected through cash, bank transfers, or third -party applications, and records are maintained using paper files, registers, or spreadsheet - based tools. These records are often stored independently by different departments or individuals, leading to data inconsistency and duplication. The absence of a centralized system makes it difficult to maintain accurate and update payment information.

In the current system, payment verification is a slow and tedious process. Service providers must manually check receipts, bank statements, or transaction confirmations to verify whether a payment has been successfully completed. This process consumes significant time and effort, especially when dealing with large number of customers.Additionally,customers do not have realtime visibility into their payment status, which often leads to confusion, repeated inquiries, and dissatisfaction.

Another major issue with the existing system is the lack of proper security mechanisms. User authentication is either weak or completely absent, which increases the risk of unauthorized access and data misuse. Sensitive information such as customer details and payment records is vulnerable to loss or manipulation. Moreover, report generation in the existing system is completely manual and requires extensive effort to calculate totals, analyse transactions, and prepare summaries. Overall, the existing payment system is inefficient, error-prone, and unsuitable for modern digital requirements.

3.1.1. DRAWBACKS

One of the major drawbacks of the existing payment system is its heavy dependency on manual processes. Most payment records are maintained using paper documents or basic spreadsheet tools, which require continuous human involvement. This increases the chances of errors such as incorrect data entry, miscalculation of amounts, and duplication of records. Even a small mistake in recording payment details can lead to serious financial discrepancies and customer dissatisfaction.Manual handling also make system slow and inefficient, especially when the number of transactions increases.

Another significant drawback is the absence of a centralized database. In the existing system, payment data is often stored in multiple locations or maintained separately by different departments or individuals. This leads to data inconsistency and makes it difficult to retrieve accurate information when required. Searching for old payment records or transaction history becomes time – consuming and sometimes impossible. The lack of centralized storage also affects coordination between service providers and customers, resulting in confusion and delays.

The existing system also lacks real–time payment tracking and verification. The Payment confirmation usually requires manual checking of receipts or bank statements, Which consumes a lot of time. Customers are not able to instantly verify whether their payment has been successfully processed. This delay often results in repeated follow - ups misunderstandings, and poor user experience. Without real-time updates, both customers and service providers face uncertainty regarding payment status.

Report generation and data analysis are also inefficient in the existing system. Financial reports such as daily, monthly, or yearly payment summaries must be prepared manually, which is time -consuming and prone to errors. Analyzing transaction trends or identifying pending payments becomes difficult due to the lack of automated tools. This affects decision-making and reduces the ability of management to monitor financial performance effectively.

3.2 Proposed System

The proposed Payment System is a Java Full Stack web-based application designed to overcome the limitations of the existing system. It provides a centralized platform where all payment-related activities are automated and managed efficiently.The system allows customer to register, log in securely, view available services, make payments online, and track their transaction history in real time. Service providers can manage services, monitor payments receive and maintain customer records through a single integrated system.

The backend of the proposed system is developed using the Spring Boot framework, which handles business logic, user authentication, authorization, and session management. A layered architecture is followed, separating the controller, service, and repository layers to improve maintainability and scalability.MySQL is used as the relational database to store user data, service details, and payment records in a structured and secure manner, ensuring data integrity and consistency.

3.2.1 Advantages

The proposed Payment System offers several advantages over the existing system, making it a reliable and efficient solution for payment management.

* Fully automated and centralized payment processing

*Faster and more accurate transaction handling

* Secure authentication and session management

*Real-time payment status and transaction tracking

*Reduced manual effort and minimized human errors

*Improved data security and integrity

*Easy generation of payment reports and summaries

*Enhanced transparency and user satisfaction

CHAPTER – 4

REQUIREMENTANALYSIS

4.1 Hardware Requirements:

- **Processor:** Intel i3 or higher
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Hard Disk:** Minimum 20 GB free space
- **System:** 64-bit system
- **Network:** Stable internet connection (for client-server access)

4.2 Software Requirements:

*Operating System: Windows 10 or above / Linux / macOS

* Programming Language: java

* Backend Framework: Spring Boot

*Frontend Technologies

- JSP (Java Server Pages)
- HTML, CSS
- Bootstrap

* Database: MySQL

* Web Server: Apache Tomcat (embedded or external)

* Build & Dependency Tool: Maven

* Development Environment: Eclipse IDE / IntelliJ IDEA / Spring Tool Suite (STS)

*Web Browser: Google Chrome / Mozilla Firefox

CHAPTER - 5

SOFTWARE DESIGN

5.1 System Architecture

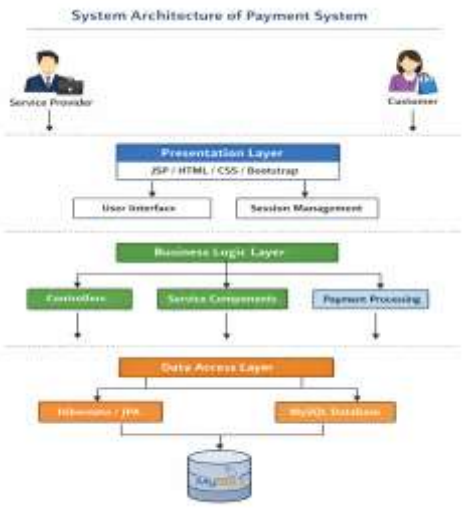
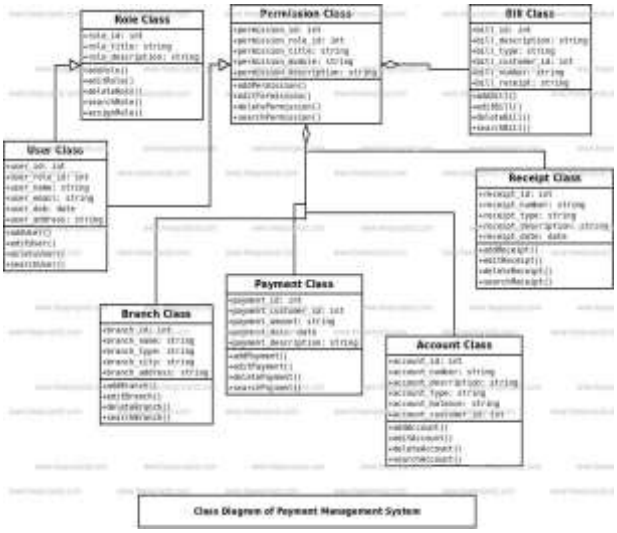


Fig 5.1 System Architecture

5.2 UML Diagrams

5.2.1 Class Diagram

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is - a" or "has - a" relationship.Each class in the class diagram may be capable of providing certain functional.These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely.

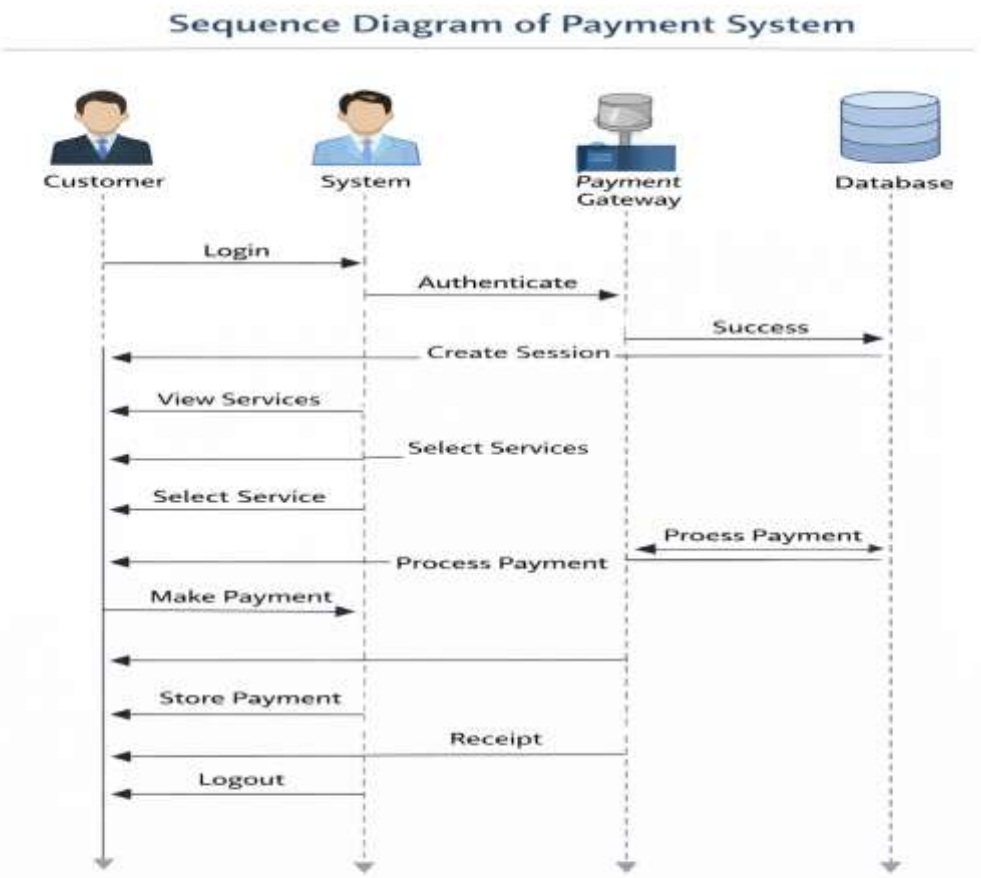


5.2.2 Use Case Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functional provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram

5.2.3 Sequence Diagram

A sequence diagram represents the interaction between different objects in the system. The important aspect of a sequence diagram is that it is time - ordered. This means that the exact sequence of the interactions between the objects is represented step by step. Different objects in the sequence diagram interact with each other by passing "messages"



Sequence Diagram of Payment System

CHAPTER - 6

SYSTEM ENVIRONMENT

6.1 What is Java:

Java is a high - level, object - oriented, platform – independent programming language developed by Sun Microsystems (now owned by Oracle).Java follows the principle "Write Once,Run Anywhere (WORA)", which allows applications to run on any platform that supports the Java Virtual Machine (JVM)

In the payment system, Java is primarily used for:

- Backend development
- Payment transaction processing
- User authentication and authorization
- Secure API development
- Business logic implementation

Java frameworks such as **Spring Boot** and **Hibernate** are used to build robust and secure payment applications.

6.2 Advantages of Java over Other Language:

***Platform Independence**
Java applications can run on multiple operating systems without modification.

***High Security**
Java offers built-in security features such as encryption,authentication,and access control,which are critical for payment systems

***Object-Oriented Approach**
Java's object-oriented nature helps in developing modular, reusable, and maintainable code

***Scalability**
Java is highly scalable and suitable for handling large volumes of payment transactions

***Rich Framework Support**
Frameworks like Spring Boot,Spring Security,and Hibernate simplify development and enhance system performance.

***Strong Community Support**
Java has extensive documentation and community support, making development easier.

6.3 Disadvantages of Java

Despite its strengths, Java has certain limitations:

***High Memory Consumption**
Java applications require more memory due to JVM overhead.

***Slower Startup Time**
Java applications may take longer to start compared to lightweight scripting languages.

***Complexity**
Java code can be more verbose, increasing development time.

***Performance Overhead**
The use of JVM can introduce performance overhead in realtime systems if not optimize.

These disadvantages can be minimized using optimized frameworks and efficient coding practices.

6.4 History of Java

Java was developed by **James Gosling** at Sun Microsystems and released in **1995**. It was designed to be secure, robust, and platform-independent.

Key milestones in Java history include:

* Java 1.0 – Initial release

*Java 2 (J2SE, J2EE) – Introduction of enterprise features

* Java SE 8 – Introduction of lambda expressions

* Java SE 17+ – Long-term support versions with enhanced performance

Java is widely used in **banking and financial applications**, including online payment systems and transaction processing platforms.

6.5: What is Spring Boot

Spring Boot is an **open-source Java-based framework** used to create **standalone,**

- Handling payment transactions
- Creating RESTful APIs
- Integrating databases such as MySQL
- Implementing authentication and authorization
- Managing business logic efficiently

Spring Boot includes an **embedded server** such as Tomcat, which allows applications to run independently

Advantages of Spring Boot

Spring Boot offers several advantages that make it ideal for payment system development:

Auto Configuration
Automatically configures application components based on project dependencies, reducing manual setup.

Faster Development
Enables rapid application development with minimal boilerplate code.

Embedded Servers

Comes with embedded Tomcat,Jetty,or Undertow,eliminating the need for external servers.

Microservices Support
Well-suited for building scalable microservices-based payment systems.

Disadvantages of Spring Boot

Despite its advantages, Spring Boot has certain limitations:

1. High Memory Consumption

Embedded servers and auto-configuration increase memory usage.

2.Limited Control Over Configuration

Auto-configuration may reduce fine-grained control over application behaviour.

3.Steep Learning Curve for Beginners

Understanding Spring concepts such as dependency injection and annotations can be challenging.

4. Long Startup Time

Spring Boot applications may take longer to start compared to lightweight frameworks.

5.Not Ideal for Small Applications

May be excessive for very small or simple projects.

History of Spring Boot

Spring Boot was introduced by **Pivotal Software** in **2014** as an extension of the Spring Framework. It was designed to simplify Spring application development by eliminating complex XML configurations.

Key milestones in Spring Boot history include:

* **2014** – Initial release of Spring Boot

***Spring Boot 1.x** – Simplified configuration and embedded servers

***Spring Boot 2.x** – Improved performance and enhanced production features

***Spring Boot 3.x** – Supports Java 17+, improved native compilation, and modern enterprise standards

Today, Spring Boot is widely used in **banking,fintech,and payment gateway systems** due to its robustness, scalability, and security features.

6.6: What is JSP (Java Server Pages)

Java Server Pages (JSP) is a **server-side web technology** used to create **dynamic web pages** in Java - based applications. JSP allows developers to embed Java code directly into HTML pages using special JSPtags.It runs on the server and generates dynamic content before sending it to the client's browser.

In a **payment system**, JSP is mainly used for:

- Designing user interfaces for payment portals
- Displaying transaction details
- Handling form submissions
- Interacting with backend services
- Creating dynamic dashboards for users and administrators

JSP works closely with **Servlets**, where JSP handles the presentation layer and Servlets business logic.

Advantages of JSP

JSP provides several advantages in Java Full Stack payment system development:

*** Easy Integration with Java**

JSP seamlessly integrates with Java classes, Servlets, and Spring Boot backend.

*** Dynamic Content Generation**
Allows dynamic generation of payment details such as transaction status and receipts.

***Separation of Presentation Layer**
Helps separate UI design from backend logic, improving maintainability.

***Reusable Components**
Supports reusable components through JSP tags and JavaBeans.

***Platform Independent**
JSP applications run on any server that supports Java.

***Fast Development**
Reduces development time by simplifying server-side page creation.

Disadvantages of JSP

Despite its usefulness, JSP has some limitations:

1. **Mixing of Java and HTML**
   Embedding Java code in HTML can reduce readability and maintainability.

2. **Performance Overhead**
   JSP pages are compiled into Servlets, which may cause slight overhead.

3. **Not Ideal for Large Applications**
   Managing large applications using JSP alone can be complex.

4. **Limited Frontend Capabilities**
   JSP lacks modern frontend features compared to frameworks like React or Angular.

5. **Debugging Complexity**
   Debugging JSP pages can be more difficult than pure Java code.

History of JSP

Java Server Pages was developed by **Sun Microsystems** and released in **1999** as part of the Java EE platform. It was designed to simplify the creation of dynamic web content compared to traditional Servlets.

Key milestones in JSP history include:

- **1999** – Introduction of JSP 1.0
- **JSP 2.0** – Introduction of Expression Language (EL) and tag libraries
- **JSP 2.3** – Improved performance and compatibility
- **Jakarta Server Pages** – Transition to the Jakarta EE platform
-

JSP continues to be used in **enterprise and payment-based applications**, especially where server -side rendering is required.

6.7: What is Bootstrap

Bootstrap is a **free, open-source front-end framework** used for developing **responsive and mobile-First web applications**. It is primarily based on **HTML, CSS, and JavaScript** and provides pre-design components such as buttons, forms, tables, navigation bars, and modals.

In a **payment system**, Bootstrap is used to:

- Design responsive payment interfaces
- Create user-friendly forms for payment details
- Build dashboards for users and administrators
- Ensure consistent UI across different devices
- Improve overall user experience

Bootstrap helps developers create visually appealing and consistent user interfaces with minimal effort.

Advantages of Bootstrap

Bootstrap offers several advantages for payment system development:

1. **Responsive Design**
   Automatically adapts the layout to different screen sizes such as mobile, tablet, and desktop.
2. **Pre-Built UI Components**
   Provides ready-made components like forms, buttons, alerts, and tables, speeding up development.
3. **Consistency Across Browsers**
   Ensures uniform appearance across different web browsers.
4. **Easy Integration**
   Easily integrates with JSP, HTML, and JavaScript in Java Full Stack applications.
5. **Faster Development**
   Reduces the need for writing custom CSS code.
6. **Customization Support**
   Allows developers to customize themes and styles according to project requirements.

Disadvantages of Bootstrap

Despite its benefits, Bootstrap has some drawbacks:

1. **Limited Design Uniqueness**
   Websites using Bootstrap may look similar without customization.

2. **Large File Size**
   Including all Bootstrap features may increase page load time.

3. **Learning Curve for Beginners**
   Understanding Bootstrap classes and grid system may take time.

4. **Over-Dependence on Framework**
   Developers may rely too much on Bootstrap instead of writing custom styles.

5. **Unused Components**
   Many features may remain unused, increasing resource usage.

History of Bootstrap

Bootstrap was developed by **Mark Otto and Jacob Thornton** at **Twitter** and was released in **2011**. It was initially created to maintain consistency in Twitter's internal tools.

Key milestones in Bootstrap history include:

- **2011** – Initial release of Bootstrap
- **Bootstrap 3** – Mobile-first design approach
- **Bootstrap 4** – Flexbox support and improved components
- **Bootstrap 5** – Removal of jQuery dependency and enhanced performance

Bootstrap is widely used in **web - based applications**, including **online payment systems**, due to its simplicity, responsiveness, and ease of use.

6.8: APPLICATIONS OF JAVA

Java is one of the most widely used programming languages in the world due to its **platform independence, security, robustness, and scalability**. Since its introduction, Java has been extensively adopted across various domains ranging from desktop applications to large-scale enterprise systems. Its ability to run on different platforms using the **Java Virtual Machine (JVM)** makes it a preferred choice for developers and organizations.
Java plays a crucial role in modern software development, especially in **banking, finance, payment systems, web applications, and enterprise solutions**.

1. Web Applications
Java is extensively used in developing **dynamic and secure web applications**. Technologies such as **Servlets, JSP (Java Server Pages), Spring, Spring Boot, and Hibernate** enable developers to create Scalable and maintainable web solutions.
Applications include:

- Online payment systems
- E-commerce websites
- Banking portals
- Online booking systems
- Educational management systems

Java-based web applications are known for their **high performance, security, and reliability**, which are essential for handling sensitive user and transaction data.

2. Enterprise Applications
Java is a dominant language in **enterprise-level application development**. Large organizations use Java to build complex business systems due to its ability to handle **high traffic, large databases, and distributed systems**.
Examples include:
- Banking and financial systems
- Insurance management systems
- Enterprise Resource Planning (ERP) systems
- Customer Relationship Management (CRM) systems

Frameworks like **Spring, Spring Boot, and Java EE** simplify enterprise application development and ensure better scalability and maintainability.

3. Mobile Applications (Android Development)

Java is one of the primary languages used for **Android application development**. Android SDK is largely based on Java, making it easier for developers to build feature - rich mobile applications.

Applications include:

- Mobile banking apps
- Digital wallet applications
- Payment and UPI apps
- Social networking apps
- Utility and productivity apps

Java strong memory management and security features make it suitable for mobile platforms handling sensitive information.

4. Desktop Applications

Java is used to develop **cross-platform desktop applications** using technologies like **Java AWT, swing, and JavaFX**. These applications can run on Windows, Linux, and macOS without modification.

Examples include:

- Accounting software
- Billing systems
- Inventory management systems
- Desktop banking tools
- Educational software

Java desktop applications are known for their **portability and stability**.

5. Banking and Financial Applications

Java is widely used in the **banking and financial sector** due to its robustness, security, and reliability. Most banking applications require handling large volumes of transactions securely, which Java supports effectively.

Applications include:

- Core banking systems
- Online payment gateways
- ATM software
- Credit card processing systems
- Fraud detection systems

Java's encryption libraries and secure APIs make it ideal for handling confidential financial data.

6. Payment Systems and FinTech Applications

Java plays a vital role in payment systems and financial technology (FinTech) applications. Its ability to integrate with databases,APIs,and security frameworks makes it suitable for transaction-based systems.
Applications include:

- Online payment processing systems
- Digital wallets
- Transaction management systems
- Fraud detection using machine learning
- Real-time transaction monitoring
-

Java frameworks like Spring Boot and Spring Security are commonly used to develop secure payment environments.

7. Cloud-Based Applications

Java is widely used in developing cloud-based applications and services. Many cloud platforms support Java-based applications due to their scalability and flexibility.
Applications include:

- Cloud-hosted enterprise systems
- SaaS (Software as a Service) applications
- Microservices-based architectures
- Distributed systems
-

Java integrates easily with cloud services such as AWS, Azure, and Google Cloud.

8. Big Data Technologies

Java is heavily used in Big Data technologies. Many popular big data tools are either written in Java or support Java-based development.
Examples include:

- Apache Hadoop
- Apache Spark
- Apache Kafka
- Apache HBase

Java helps process and analyze large volumes of data efficiently, making it suitable for data-intensive applications.

6.9: Java Development Steps

1. Requirement Analysis

This step involves understanding the project objectives, functional features, and system constraints. It helps define the scope and purpose of the Java application.

2. System Design

The application architecture, database structure, and class design are planned. This step ensures proper organization of modules and smooth data flow.

3. Environment Setup

Required software such as JDK, IDE, frameworks, and databases are installed and configured. a proper setup ensures efficient development.

4. Coding and Implementation

Java code is written based on the design using classes and methods. Business logic and application features are implemented in this stage.

5. Compilation and Execution

The Java source code is compiled into bytecode and executed using the JVM. Errors are detected and corrected during execution.

6. Testing

Different testing techniques are used to verify functionality and performance. This step ensures the application works as expected.

7. Deployment

The completed application is deployed on a server or cloud platform. It is made accessible to users in the production environment.

CHAPTER – 7

MODULES USED IN PROJECT

A payment system is divided into several functional modules to ensure secure, efficient, and reliable transaction processing. Each module handles a specific responsibility within the system.

1. User Management Module

This module manages user-related activities such as registration, login, and profile management. It ensures secure authentication and authorization of users using credentials and role-based access.

Functions:

- User registration and login
- Profile management
- Password encryption and validation

2. Authentication and Authorization Module

This module ensures that only authorized users can access the system. It validates user identity and controls access to system resources.

Functions:

- Login authentication
- Role-based access control
- Session management

3. Payment Processing Module

This is the core module of the payment system. It handles payment initiation, processing, and completion.

Functions:

- Payment request handling
- Transaction processing
- Payment confirmation

4. Transaction Management Module

This module records and manages all payment transactions. It ensures accuracy, consistency, and traceability.

Functions:

- Transaction creation
- Transaction history tracking
- Status updates (success, pending, failed)

5. Payment Gateway Integration Module

This module connects the system with external payment gateways such as banks, UPI, credit/debit cards, or digital wallets.

Functions:

- Secure gateway communication
- API integration
- Response handling

6. Fraud Detection and Security Module

This module detects suspicious or fraudulent activities using security rules or machine learning techniques.

Functions:

- Fraud detection
- Transaction validation
- Risk analysis

7. Database Management Module

This module handles data storage and retrieval operations securely.

Functions:

- Storing user and transaction data
- Data consistency and backup
- Secure database access

8. Notification Module

This module sends notifications to users regarding transaction updates.

Functions:

- Email notifications
- SMS alerts
- Payment confirmation messages

9. Admin Management Module

This module allows administrators to monitor and control system operations.

Functions:

- User management
- Transaction monitoring
- Report generation

10. Reporting and Analytics Module

- Transaction reports
- User activity analysis
- Financial summaries

7.1 Install JAVA Step-by-Step in Windows or MAC

• A. Install Java on Windows

Step 1: Download JDK

1. Open a web browser.
2. Go to the official Oracle website: https://www.oracle.com/java/technologies/downloads/
3. Download the **Windows x64 Installer (.exe)** for the latest JDK version.

Step 2: Run the Installer

1. Double-click the downloaded .exe file.
2. Click **Next** in the setup wizard.
3. Choose the installation folder (default is recommended).
4. Click **Next** and wait for installation to complete.
5. Click **Close** after successful installation.

Step 3: Set Environment Variables

1. Open **Control Panel → System → Advanced system settings**.
2. Click **Environment Variables**.
3. Under **System Variables**, click **New**:
   - Variable Name: JAVA_HOME
   - Variable Value: C:\Program Files\Java\jdk-<version>
4. Select **Path → Edit → New** → add: C:\Program Files\Java\jdk-<version>\bin
5. Click **OK** to save changes.

Step 4: Verify Installation

1. Open **Command Prompt**.
2. Type:
3. java -version
4. If Java version is displayed, installation is successful.

7.2 HOW TO INSTALL JAVA ON WINDOWS

Step 1: Understanding Java Components (JDK, JRE, JVM)

Before installing Java, it is important to understand its main components. Java consists of three major parts:Java Development Kit (JDK), Java Runtime Environment (JRE), and Java Virtual Machine (JVM). The Java Development Kit (JDK) is required for developing Java applications. It includes the compiler, libraries, and development tools. The Java Runtime Environment (JRE) is used to run Java applications and contains the JVM and standard libraries. The Java Virtual Machine (JVM) converts Java bytecode into machine-readable code, enabling platform independence.

For development purposes, especially for projects like payment systems or web applications, installing the JDK is mandatory.

Step 2: Checking System Requirements

Before downloading Java,the system requirements must be verified. Java requires a compatible Windows operating system such as Windows 10 or Windows 11. The system should have sufficient disk space (at least 1–2 GB free space) and administrator privileges to install software.

It is also important to check whether the system architecture is 64-bit, as most modern Java versions support 64-bit systems. This ensures smooth installation and better performance.

.

Step 3: Downloading the Java Development Kit (JDK)

The first practical step in installing Java is downloading the Java Development Kit from the official source.Using official sources ensures security and authenticity. Open a web browser and navigate to the official Oracle Java website. On the Java Downloads page, various versions of Java are displayed. Select the latest Long-Term Support (LTS) version for stability and long-term use.

Under the Windows section, choose the Windows x64 Installer (.exe) file. This installer is specifically designed for Windows systems and provides an easy installation process. Once selected, accept the license agreement and download the installer file to the system.

Step 4: Running the Java Installer

After the download is complete, locate the installer file in the Downloads folder. Double-click the **.exe** file to start the installation process. The Java Setup Wizard will open, displaying a welcome screen. Click the **Next** button to proceed. The installer will prompt the user to choose the installation directory. By default, Java is installed in the Program Files directory, which is recommended for most users. Advanced users may change the location, but it is generally advisable to keep the default path

Step 5: Completing the Installation Process

During installation, the setup wizard shows a progress bar indicating the installation status. This process may take a few minutes depending on system performance.

After completion, a confirmation message is displayed indicating that Java has been successfully installed. Click the **Close** button to exit the installer. At this stage, Java is installed on the system but may not yet be accessible from the command line.

Step 6: Configuring Environment Variables

To use Java from the Command Prompt, environment variables must be configured. Environment variables help the operating system locate Java executables easily.

Open **Control Panel**, navigate to **System**, and click on **Advanced system settings**. In the System Properties window, click on the **Environment Variables** button.

Under **System Variables**, click **New** and create a variable named JAVA_HOME. Set its value to the Java installation path, for example:

C:\Program Files\Java\jdk-17

Next, locate the **Path** variable under System Variables and click **Edit**. Add a new entry pointing to the Java bin directory:

C:\Program Files\Java\jdk-17\bin
Click **OK** to save all changes. These steps ensure Java commands can be executed from any directory.

Step 7: Importance of Environment Variables

Setting environment variables is crucial for Java development.Without proper configuration the system may not recognize Java commands such as java or javac.

The JAVA_HOME variable is used by development tools, IDEs, and frameworks like Spring Boot to locate the Java installation. The Path variable ensures that Java commands can be run directly from the command line without specifying the full path.

Proper configuration avoids runtime errors and ensures compatibility with development environments.

Step 8: Verifying Java Installation

After installation and configuration, Java must be verified to ensure it is working correctly. Open the **Command Prompt** by pressing Win + R, typing cmd, and pressing Enter.
In the Command Prompt window, type the following command:
java -version
Press Enter. If Java is installed correctly, the system will display the installed Java version along with runtime environment details. This confirms that Java is properly installed and configured.

Step 9: Troubleshooting Common Issues

Sometimes, users may face issues during or after installation. Common problems include Java not recognized as a command or incorrect version display.
These issues usually occur due to incorrect environment variable configuration. Rechecking The Path and JAVA_HOME values and restarting the system often resolves the problem. Ensuring no older Java versions conflict with the new installation is also important.

Step 10: Java Installation for Development Use

Once Java is installed successfully, it can be used for development purposes. Developers Can install IDEs such as **Eclipse**, **IntelliJ IDEA**, or **NetBeans** to write and execute Java programs.

Java installation is a foundational step for building applications such as **web applications, enterprise systems, payment systems, and Android applications**.

INSTALLATION OF ECLIPSE IDE

Eclipse is a widely used **open-source Integrated Development Environment (IDE)** mainly used developing Java applications. It provides powerful tools for writing, compiling, debugging, and running Java programs. Eclipse is commonly used for developing **Java Full Stack applications**, including web applications and payment systems.

Step 1: Prerequisite – Java Installation

Before installing Eclipse, **Java must be installed** on the system. Eclipse requires the **Java Development Kit (JDK)** to function properly. After installing Java,verify it by running the command java -version in the Command Prompt. Once Java is successfully installed, Eclipse can be installed.

Step 2: Download Eclipse Installer

1. Open a web browser on your Windows system.
2. Go to the official Eclipse website:
   https://www.eclipse.org/downloads/
3. Click on the **Download** button for *Eclipse Installer*.
4. The installer file (eclipse-inst-jre-win64.exe) will be downloaded to your system.

Downloading Eclipse from the official website ensures security and access to the latest features.

Step 3: Run the Eclipse Installer

1. Navigate to the Downloads folder.
2. Double-click the downloaded **Eclipse Installer** file.
3. If a security warning appears, click **Yes** to allow the installer to run.
4. The Eclipse Installer window will open and display different Eclipse packages.

Step 4: Choose Eclipse IDE for Java Developers

From the list of available packages:

1. Select **Eclipse IDE for Java Developers**.
2. This package includes tools required for Java SE development.
3. For web or enterprise projects, *Eclipse IDE for Enterprise Java and Web Developers* can also be selected.

Choosing the correct package ensures proper tools and libraries are installed.

Step 5: Select Installation Folder

1. The installer will prompt for an installation directory.
2. By default, Eclipse is installed in the **User directory**, which is recommended.
3. You can change the location if required.
4. Click **Install** to begin the installation process.

Step 6: Accept License Agreement

1. Read the Eclipse license agreement.
2. Click **Accept Now** to continue.
3. The installer will start downloading and installing required files.

This process may take a few minutes depending on internet speed.

Step 7: Complete Installation

After installation:

1. A success message will be displayed.
2. Click **Launch** to open Eclipse IDE.
3. Eclipse will now start for the first time.

Step 8: Select Workspace

When Eclipse launches:

1. It asks to select a **Workspace**.
2. A workspace is a folder where all Java projects are stored.
3. Choose a default location or browse to a new folder.
4. Click **Launch** to continue.

Step 9: Verify Eclipse Installation

Once Eclipse opens successfully:

1. Create a new Java Project.
2. Write a simple Java program.
3. Run the program to confirm Eclipse is working properly.

Successful execution confirms proper installation.

CHAPTER – 8
TESTING

8.1 System Test

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent ensuring that the Software system meets requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

8.2 Types Of Tests

8.2.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic Is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests are basic tests at component level test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

8.2.2 Integrated Testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components correctly and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

8.2.3 Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

1. **Valid Input:** Identified classes of valid input must be accepted.
2. **Invalid Input:** Identified classes of invalid input must be rejected
3. **Functions:** Identified functions must be exercised.
4. **Output:** Identified classes of application outputs must be exercised.
5. **Systems/Procedures:** Interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

8.2.4 White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure, and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

8.2.5 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. you cannot "see" into it. The test provides inputs and responds to outputs without considering how the software works.

UNIT TESTING:

- Test strategy and approach
- Field testing will be performed manually and functional tests will be written in detail.
- Test objectives
- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.
- Features to be tested
- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.
- Integration Testing
- Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.
- The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.
- Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

- User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.
- Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Test Case – 1:

Test case for Login form:

FUNCTION:	LOGIN
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	No

HIGH PRIORITY	Yes

Test Case – 2:

Test case for User Registration form:

FUNCTION:	USER REGISTRATION
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test Case – 3:

- Test case for Change Password:
- When the old password does not match with the new password ,then this results in displaying an error message as " OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD".

FUNCTION:	Change Password
EXPECTED RESULTS:	Should check if old password and new password fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

CHAPTER - 9
IMPLEMENTATION

9.1 Methodology

METHODOLOGY OF PAYMENT SYSTEM

The methodology of the payment system defines the systematic approach used to design, develop and implement a secure, reliable, and efficient payment processing application. The proposed payment system follows a **modular, layered, and service - oriented architecture**, ensuring scalability, security, and maintainability. The methodology will integrates modern **Java Full Stack technologies** including Spring Boot,RESTful services, database management, and secure transaction handling.

1. Requirement Analysis

The first phase of the methodology involves identifying and analyse system requirements. Functional requirements include user registration, authentication, payment initiation, transaction processing, payment confirmation, and transaction history management. requirements focus on security, performance, reliability, scalability, and availability. User requirements are gathered by understanding the needs of customers, merchants, and administrators. Special emphasis is placed on secure data handling, compliance with payment standards, and fast transaction processing.This phase ensures that the system objectives are clearly defined before moving to design and development.

2. System Design

Based on the analysed requirements, the system design phase defines the overall architecture of payment system. The system is designed using **three tier architecture**, consisting of the presentation layer, business logic layer, and data layer.

3.Technology Selection:

The payment system is developed using **Java Full Stack technologies**. Spring Boot is used for backend development due to its ease of configuration, embedded server support,And robust security features.RESTful APIs are implemented to enable connect

4. Implementation Methodology

The implementation phase follows a **modular development approach**, where each module is developed and tested independently.Major module include user management, payment processing, transaction validation, notification handling, and reporting.
Spring Boot controllers handle incoming requests, services contain business logic, and repositories manage database interactions. Security mechanisms such as authentication, authorization, data encryption are integrated to protect sensitive payment information.

5. Integration with Payment Gateway

A critical part of the methodology is integrating the system with a third-party payment gateway. Secure APIs are used to communicate with the payment gateway for transact authorization and settlement.

.

6. Testing and Validation

Testing is performed at multiple levels to ensure system reliability. Unit testing verifies individual components, while integration testing checks interactions between modules. System testing ensures the complete payment flow works as expected. Security testing is conducted to identify vulnerabilities such as unauthorized access and data breaches. Performance testing ensures the system can handle multiple concurrent transactions efficiently.

7. Deployment and Maintenance

After successful testing, the payment system is deployed on a production server Continuous monitoring is implemented to track system performance and transaction status. Regular updates and maintenance are carried out to improve security, fix bugs, and enhance system features.The system is designed to support future upgrades such as adding new payment methods,integrating analytics,scaling for increased transaction.

9.2 Source Code

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"

 pageEncoding="ISO-8859-1"%>

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<html>

<head>

<meta charset="UTF-8">

 <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <link rel="icon" type="image/x-icon" href="https://i.postimg.cc/sgHqJ95p/payment-system

-logo.png">

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css" rel=

"stylesheet" integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC" crossorigin

="anonymous">

  <title>Add Products</title>

</head>

<body>

<c:if test="${sessionScope.biller == null}">

     <div class="container">

         <div class="row justify-content-center">

             <div class="col text-center">
```

```
<a class="btn btn-primary m-2 fw-bold ps-5 pe-5" href="/home" role="button">Home Page...
 </a>
      </div>
        </div>
     </div>
</c:if>
<c:if test="${sessionScope.biller != null}">
  <nav class="navbar navbar-expand-lg navbar-dark bg-warning sticky-top border-bottom
border-5 border-white">
    <div class="container-fluid">
      <a class="navbar-brand" href="/biller/dashboard">
        <img src="https://i.postimg.cc/sgHqJ95p/payment-system-logo.png" alt="" width="40"
 height="40" class="d-inline-block">
        <span class="text fw-bold fs-4 ms-1">Payment System</span>
      </a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-expanded
="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
```

```
 </nav>

<div class="row container-fluid">

  <div class="col col-3">

    <h3 class="text-center text-primary pt-3">Biller Dashboard</h3>



    <ul class="nav flex-column">

      <li class="nav-item p-3">

        <a class="nav-link text-light fw-bold btn btn-primary p-3" href="/biller/profile">View
Profile</a>

      </li>

      <li class="nav-item p-3">

        <a class="nav-link text-light fw-bold btn btn-primary p-3" href="/biller/add-product">
Add Product</a>

      </li>

      <li class="nav-item p-3">

        <a class="nav-link text-light fw-bold btn btn-primary p-3" href="/biller/view-products">
View Products</a>

      </li>

      <li class="nav-item p-3">

      </li>
```

```
  <li class="nav-item p-3">
        <a class="nav-link text-light fw-bold btn btn-primary p-3" href="/biller/all-orders">
All Orders
</a>
      </li>
      <li class="nav-item p-3">
        <a class="nav-link text-light fw-bold btn btn-primary p-3" href="/biller/registered-
customers"
>Registered Customers</a>
      </li>
    </ul>
   </div>
   <div class="col col-9">
    <div class="message text-center text-warning fw-bold mt-3">
      ${message}
    </div>
    <div class="row m-2 container">
      <span class="text text-center fs-4 fw-bold mt-4">Add Product</span>
        <div class="d-flex justify-content-center mt-3">
          <form class="form" action="/biller/add-product" method="post">
```

```
<div class="mb-3">
    <label for="price" class="form-label">Enter Product Price</label>
    <input type="number" class="form-control" id="price" name="price" required>
</div>
  <div class="mb-2">
        <label for="description" class="form-label">Product Descriptions</label>
        <textarea class="form-control" id="description" rows="2" name="descry
        ption"    required></textarea>
        <div id="emailHelp" class="form-text text-white">Mention Product Details Briefly
       with Expire Date.</div>
       </div>
       <select class="form-select" aria-label="Default select example" name="category">
        <option selected>Chose a Category</option>
        <option value="Entertainment">Entertainment</option>
        <option value="Telecom">Telecom</option>
        <option value="Insurance">Insurance</option>
        <option value="Gas">Gas</option>
        <option value="Others">Others</option>
       </select>
  <button type="submit" class="btn btn-primary w-100 mt-3">Add Product</button>
```

```
</div>

</div>

  </div>

  </div>

</div>

<script src=https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js

integrity="sha384-MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/

tWtIaxVXM" crossorigin="anonymous"></script>

</c:if>

</body>

</html>
```

package com.techm.entity;

```
import com.techm.enums.*;

import jakarta.persistence.*;

@Entity

public class Biller {

   @Id

   @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long billerId;

 private String billerName;

 private String email;

 private String password;

 private String about;

 @Enumerated(EnumType.STRING)

 private Role role;

 @Enumerated(EnumType.STRING)

 private BillerCategory category;

 private boolean active;



 public Biller() {

 }



 public Biller(String billerName, String email, String password, String about, String category) {

    this.billerName = billerN
```

```
package com.techm.enums;

 public enum BillerCategory {

    ENTERTAINMENT("Entertainment"),

    TELECOM("Telecom"),

    INSURANCE("Insurance"),

    ELECTRICITY("Electricity"),

    WATER("Water"),

    GAS("Gas"),

    CABLE("Cable"),

    OTHER("Other");

 private final String value;

  BillerCategory(String value) {

        this.value = value;

    }

 }
```

package com.techm.repository;

```
import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;

import com.techm.entity.Biller;

@Repository

public interface BillerRepository extends JpaRepository<Biller,

findByEmailAndPassword(String email, String password

findByEmail(String email);

}
```

package com.techm.service;

```
import java.util.List;

import org.springframework.stereotype.Service;

import com.techm.entity.Biller;

import com.techm.entity.BillerDashDTO;

import com.techm.entity.Customer;
```

```
@Service

public interface BillerService {

registerBiller(Biller biller);

public Biller loginBiller(String email, String password);

public Biller getBillerById(Long billerId);

public Biller updateBiller(Biller biller);

public List<Biller> getAllBillers();

public BillerDashDTO billerMoreInfo(Long billerId);

List<Customer> registeredCustomers(Long billerId);

}
```

package com.techm.service.impl;

```
import com.techm.entity.Biller;

import com.techm.entity.BillerDashDTO;

import com.techm.entity.Customer;

import com.techm.entity.Orders;

import com.techm.enums.Role;
```

```
import com.techm.repository.BillerRepository;

import com.techm.repository.CustomerRepository;

import com.techm.service.BillerService;

import com.techm.service.OrderService;

import com.techm.service.ProductService;

import java.util.HashSet;

import java.util.List;

import java.util.Set;

import java.util.stream.Collectors;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.stereotype.Service;

@Service

public class BillerServiceImpl implements BillerService {

@Autowired

    private BillerRepository billerRepositor

@Autowired

    private CustomerRepository customerRepository;
```

```
@Autowired
    private ProductService productService;
     @Autowired
    private OrderService orderService;
@Override
    public Biller registerBiller(Biller biller) {
        String email = biller.getEmail();
        if (billerRepository.findByEmail(email) != null || customerRepository.findByEmail(email) != null) {
            return null;
        }
        biller.setRole(Role.BILLER);
        biller.setActive(true);
        return billerRepository.save(biller);
    }
```

```
@Override
    public Biller loginBiller(String email, String password) {
        return billerRepository.findByEmailAndPassword(email, password);
    }
@Override
  publicgetBillerById(Long billerId) {
  return billerRepository.findById(billerId).orElse(null);
  }
@Override
 public Biller updateBiller(Biller biller) {
  return billerRepository.save(biller);
  @Override
  public List<Biller> getAllBillers() {
  return billerRepository.findAll();
  @Override
```

```
Billebiller = getBillerById(billerId);

int totalProducts = productService.getAllProductsByBiller(biller).size();

 int totalOrders = orderService.allOrdersByBiller(billerId).size();

int pendingOrders = orderService.pendingOrdersByBiller(billerId).size();

 BillerDashDTO bdd = new BillerDashDTO();

bdd.setPendingOrders(pendingOrders);

bdd.setTotalProducts(totalProducts);

bdd.setTotalOrders(totalOrders);

}

@Override

public List<Customer> registeredCustomers(Long billerId)

List<Orders> orders = orderService.allOrdersByBiller(billerId);

Set<Customer> customers = new HashSet<>();

for(Orders order : orders) {

customers.add(order.getCustomer());

 }
```

```
package com.techm.enums;

public enum BillStatus {

    PAID("Paid"),

    PENDING("Pending"),

    OVERDUE("Overdue");



    private final String value;



    BillStatus(String value) {

        this.value = value;

    }



    public String getValue() {

        return value;

    }

}
```

CHAPTER – 10

RESULTS

Screenshots:

Payment system using spring boot and jsp:

In the Payment System, there are some service providers and customers who can buy their services And manage all the details. This is a Payment System application built using the Spring Boot framework, MySQL for database management, JSP for server-side rendering of HTML templates, and Bootstrap for styling. The project includes session tracking for secure logins and usage.

Home (Register and Login)

Home Page : When you start the spring boot application you will see the homepage of Payment System. On the home page, there are 2 options Register for new user registration and Login for existing users.

Fig 10.1: After running server it shows user login

Register Page:

When you click on Register it will show Customer Registration form. In the navigation bar, you have 2 options Register as Biller for biller registration and Login for existing users. After registration, it will redirect to the login page will showing the message *'Customer registered successfully'* and the same goes for the biller.

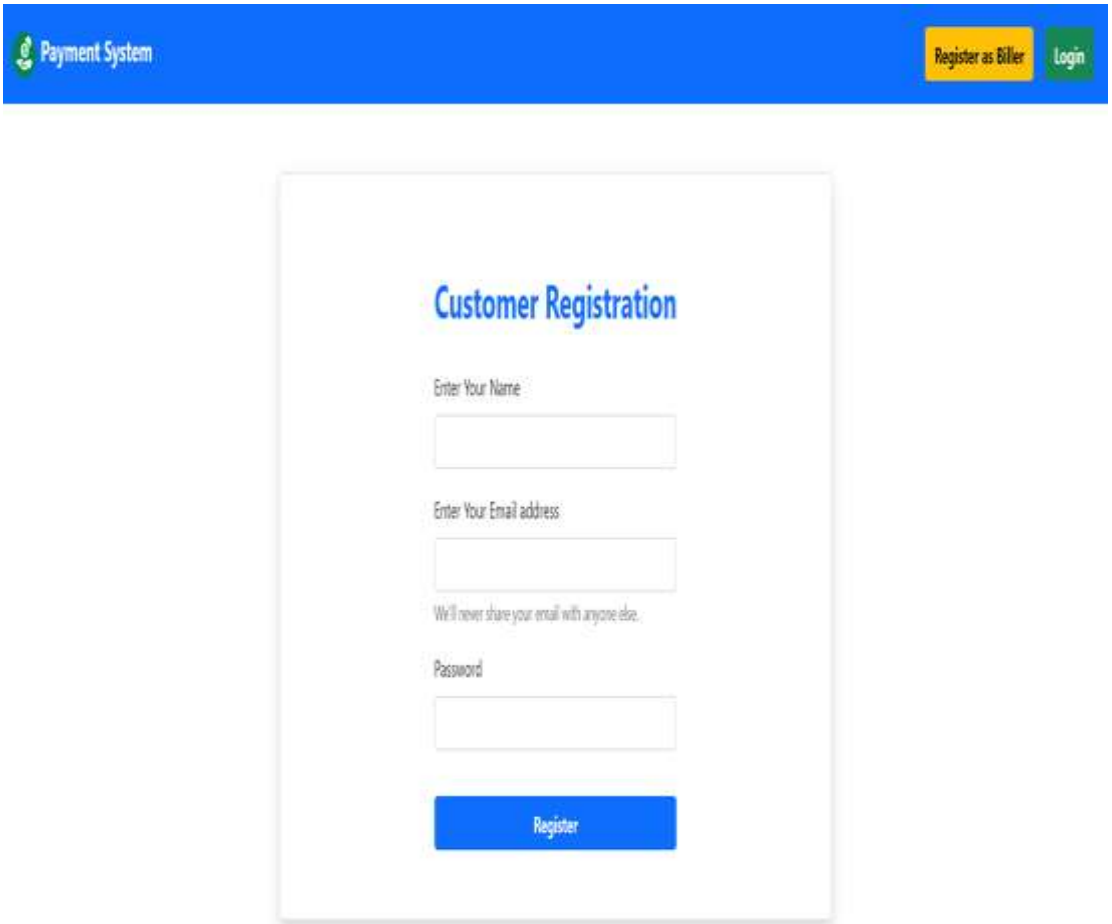


Fig 10.2 After running server it shows user login

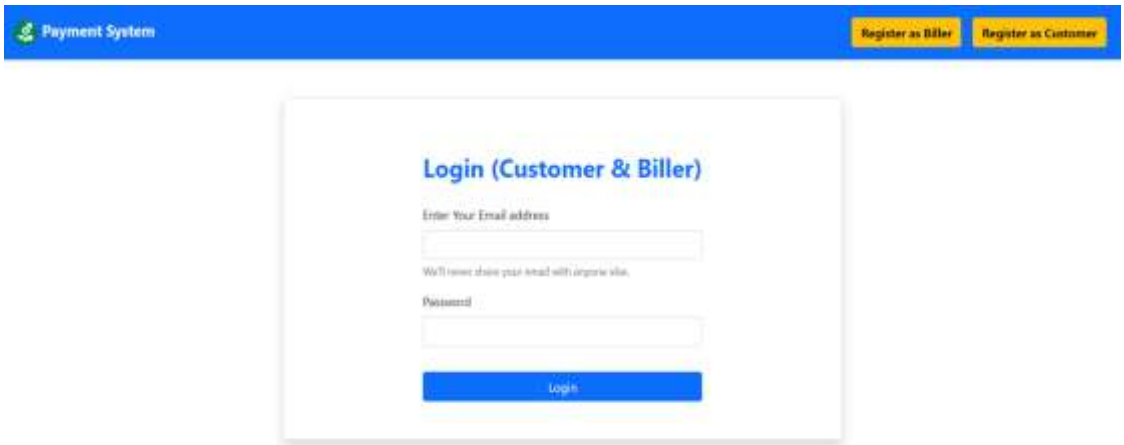


Fig 10.3 After running server it shows user login

In above screen click on 'User Login Here' link to get below page

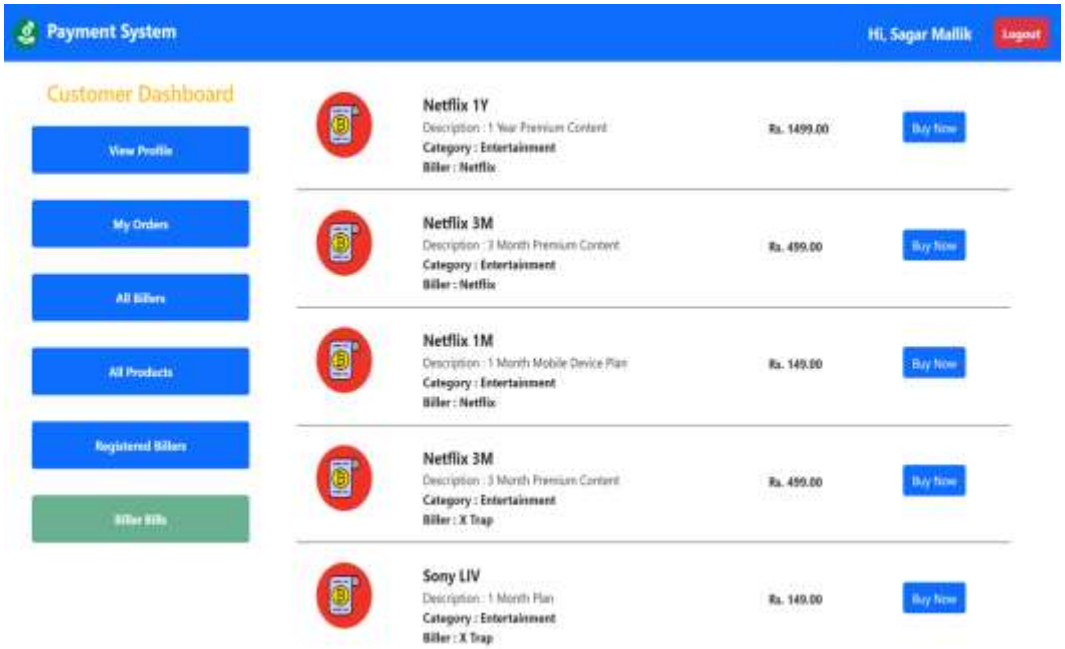


Fig 10.4 customer dashboard

Fig 10.5 customer profile

In above screen click on customer dashboard and get below page

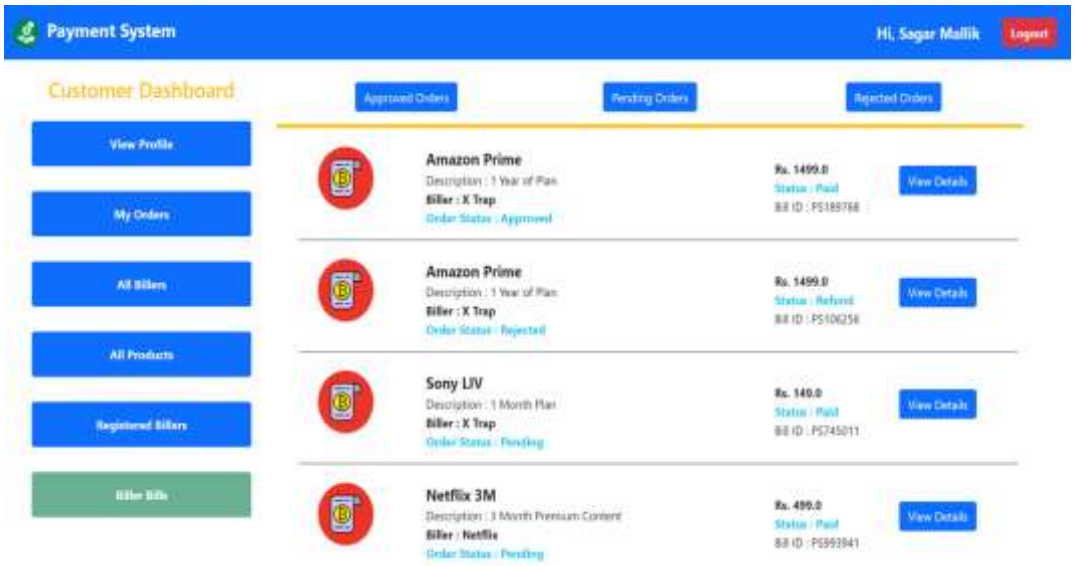


Fig 10.6 my orders

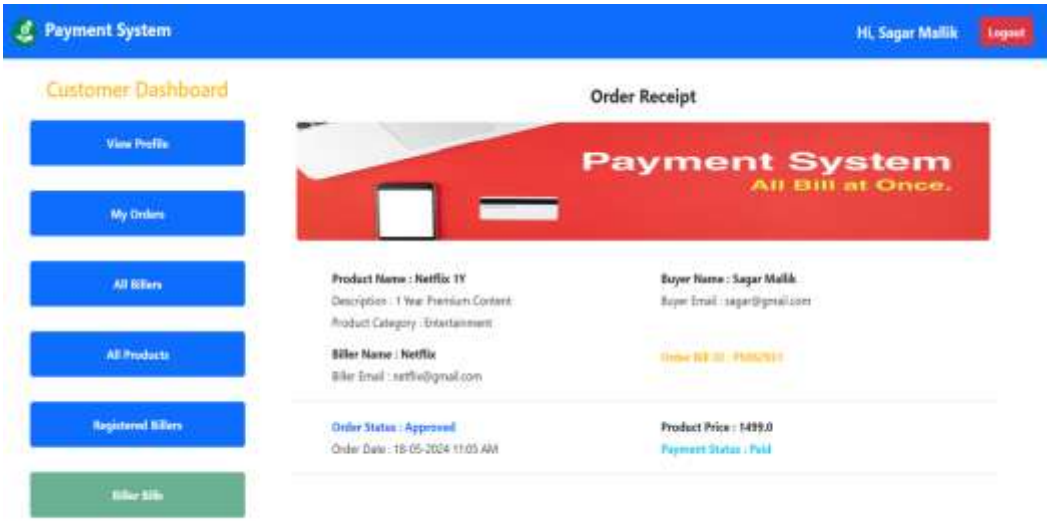


Fig 10.7 order receipts

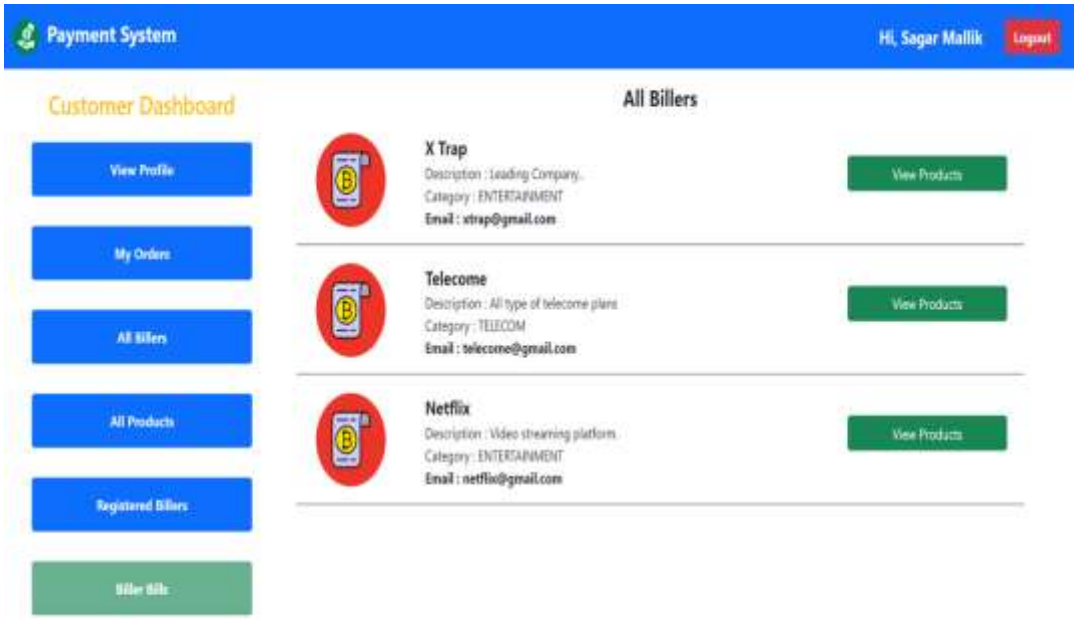


Fig 10.8 All Billers

In above screen click on 'Biller bills' to get below page to register as billers

Fig 10.9 Registered billers

In above screen selecting and uploading orders and then check the orders to get below output

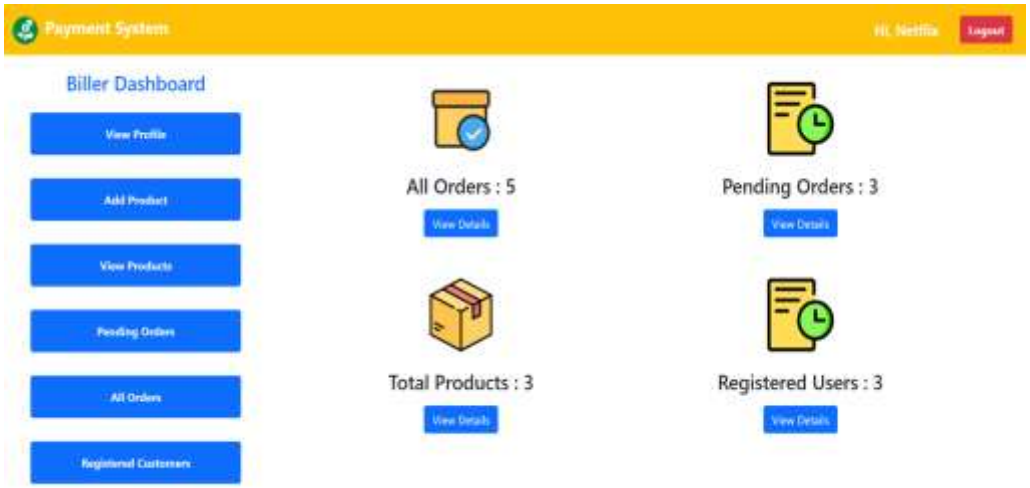


Fig 10.10 view profile of products

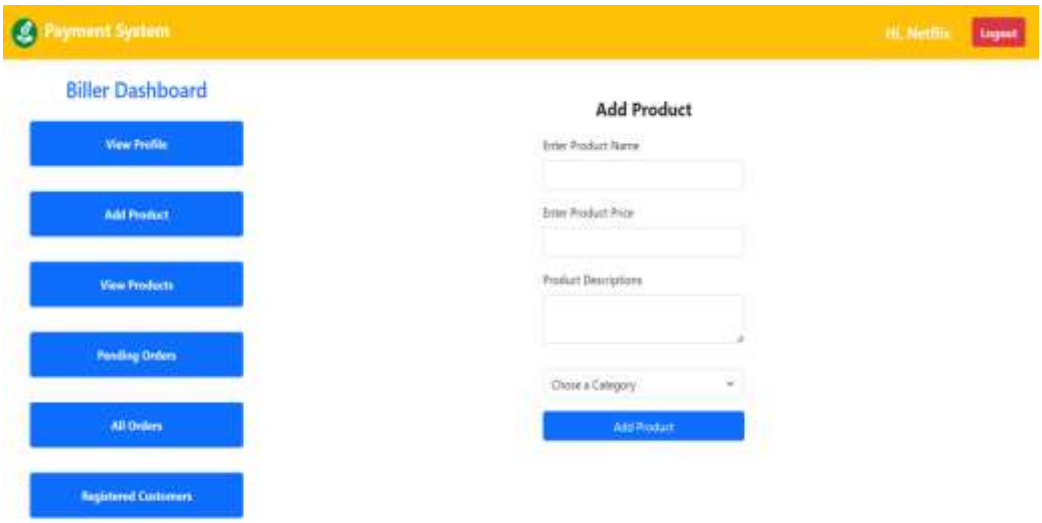


Fig 10.11 Add products

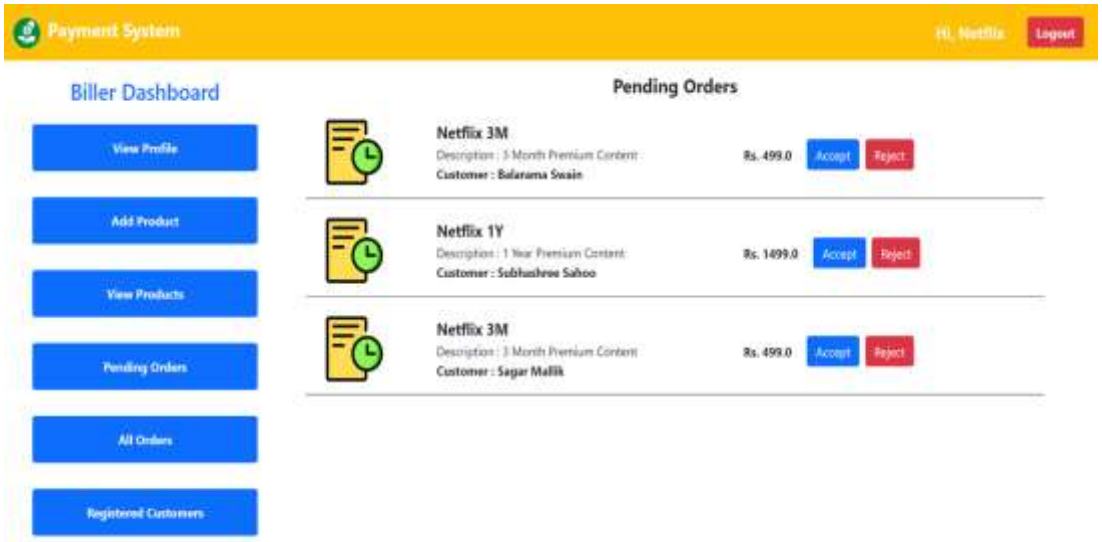


Fig 10.12 view products

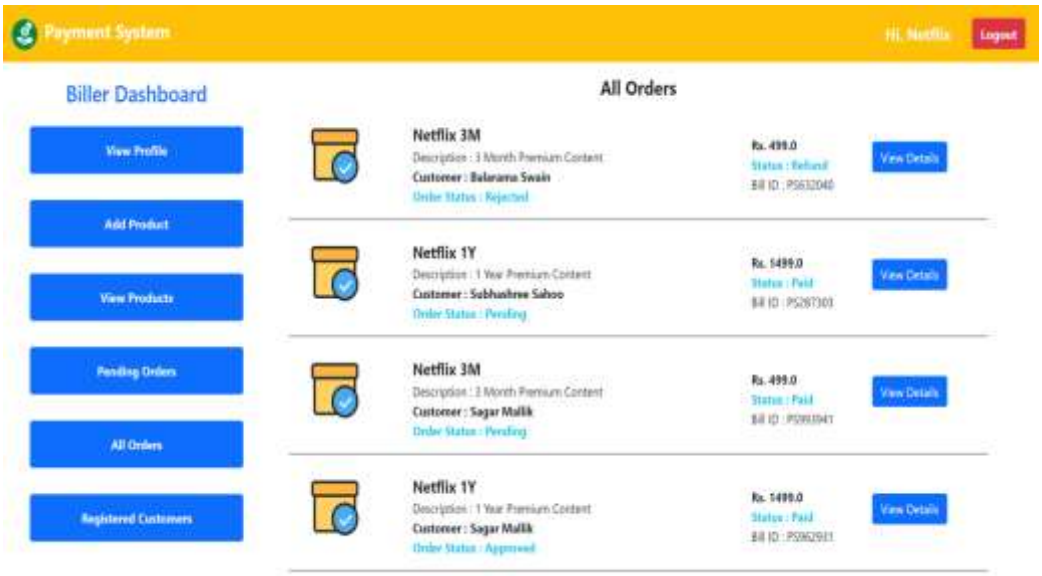


Fig 10.13 All Orders

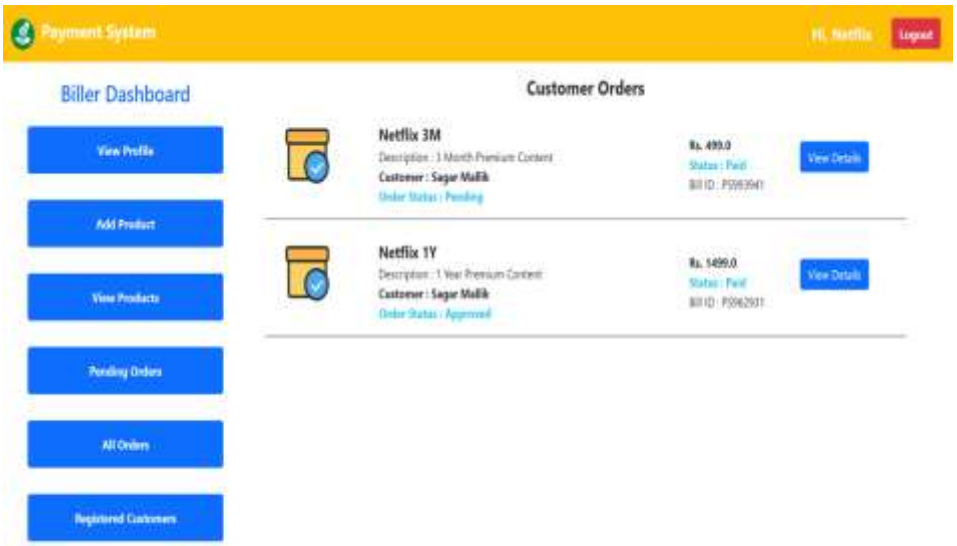


Fig 10.14 Customer orders

CHAPTER – 11
CONCLUSION

The payment system enables secure, fast, and reliable digital transactions by automating payment processes and reducing manual effort. It ensures accuracy,minimizes errors and fraud,and provides real-time transaction updates to enhance user trust. With a well-structured architecture, the system supports scalability, performance, multiple payment modes. Security features such as authentication, authorization, and data protection ensure confidentiality and integrity. Overall, the system improves efficiency, transparency, and user experience in financial operations.

In conclusion,the payment system modernizes financial service by simplifying complex transactions and ensuring stability and reliability. Its modular design allows easy maintenance and future enhance while database integration ensures accurate record management.Userfriendly interface and automate workflows reduce operational costs and processing time. The system supports business growth by enabling efficient transactions and adapting to future technological advancements.

Future Work:

In the future, payment systems are expected to become more intelligent, secure, and user -friendly by integrating advanced technologies such as artificial intelligence and machine learning. These technologies will help in real-time fraud detection, personalized payment experiences, and predictive analysis of user spending behavior. Biometric authentication methods like fingerprint scanning, facial recognition, and voice recognition will further enhance security and reduce dependency on traditional passwords or PINs. The adoption of blockchain technology can bring transparency, faster settlement, and reduced transact costs in cross-border payments. Payment systems will also support seamless integration with Internet of Things(IoT)devices,enabling automatic payments through smart devices. Increased focus will be placed on data privacy and regulatory compliance to protect user information. Cloud - based payment infrastructures will improve scalability an system availability. Digital wallets will evolve to support multiple currencies and assets. Overall, these advancements will make payments faster, safer, and more reliable.

CHAPTER – 12
REFERENCES

1. Research Papers & Articles

Recent Systematic Reviews & Literature Surveys

Chauhan & Sharma (2024) –

A Systematic Literature Review of Digital Payments — extensive review of digital payment research,methodologies, trends, and adoption factors.

EconPapers

The Emerging Technologies of Digital Payments and Associated Challenges (MDPI) - systematic overview of technologies (e-wallets, mobile payments, blockchain) and key challenges.

MDPI

The development of digital payments:past,present,and future-bibliometric study showing evolution of payment research and adoption models.

ScienceDirect

echnology-Focused Research Articles

Movva & Dasarraju (2024) – Impact of Blockchain on FinTech and Payment Systems — explores blockchain's role in enhancing security/transparency in payments.

CARI Journals

SecurePay: Enabling Secure and Fast Payment Processing for Platform Economy (arXiv 2025) — proposes a hybrid blockchain + CBDC payment processing system.

arXiv

A Stream Pipeline Framework for Digital Payment Programming (arXiv 2025) — smart contract framework for programmable digital payments.

2. GitHub Repositories

Interoperability & Payment Systems

Mojaloop (Bill & Melinda Gates Foundation) – open-source interoperable payment platform for financial inclusion. Repo: https://github.com/mojaloop/

GitHub topic: payment - system – curated list of repositories tagged payment-system (variety of projects). https://github.com/topics/payment-system

Decrypto -org/blockchain-papers – curated list of academic papers on blockchain and payments (including Lightning Network and state channels). https://github.com/decrypto-org/blockchain-papers

Bellaj / Blockchain – repository containing scanned / blockchain research scans and papers (supporting foundational blockchain research relevant to payments). https://github.com/bellaj/

Blockchain Payment Protocol / Systems Projects.

Lightning Network (Bitcoin off-chain payments) – not a GitHub repo but relevant for payment research and implementation details. GNU Taler open-source,privacy_oriented electronic payment.